

BFS

// Parallel BFS using OpenMP

// Compile: g++ -fopenmp bfs.cpp -o bfs

// Run: ./bfs

#include <iostream>

#include <vector>

#include <queue>

#include <fstream>

#include <omp.h>

using namespace std;

#define MAX_VERTICES 10000

class Graph {

public:

int V;

vector<vector<int>> adj;

Graph(int v) : V(v), adj(v, vector<int>(v, 0)) {}

void addEdge(int u, int v) {

adj[u][v] = adj[v][u] = 1;

}

void generateEdges(int edges) {

while (edges) {

int u = rand() % V, v = rand() % V;

if (u != v && !adj[u][v]) {

```

        addEdge(u, v);
        edges--;
    }
}
};

```

```

void sequentialBFS(Graph &g, int start) {

```

```

    vector<int> visited(g.V, 0);
    queue<int> q;

```

```

    visited[start] = 1;
    q.push(start);

```

```

    while (!q.empty()) {

```

```

        int node = q.front();
        q.pop();

```

```

        for (int i = 0; i < g.V; i++) {

```

```

            if (g.adj[node][i] && !visited[i]) {
                visited[i] = 1;
                q.push(i);
            }

```

```

        }
    }
}

```

```

void parallelBFS(Graph &g, int start) {

```

```
vector<int> visited(g.V, 0), q(MAX_VERTICES);
```

```
int front = 0, rear = 0;
```

```
visited[start] = 1;
```

```
q[rear++] = start;
```

```
while (front != rear) {
```

```
    int oldFront = front, oldRear = rear;
```

```
    #pragma omp parallel
```

```
    {
```

```
        vector<int> localQ;
```

```
        #pragma omp for
```

```
        for (int i = oldFront; i < oldRear; i++) {
```

```
            int node = q[i];
```

```
            for (int j = 0; j < g.V; j++) {
```

```
                if (g.adj[node][j] && !visited[j]) {
```

```
                    #pragma omp critical
```

```
                    {
```

```
                        if (!visited[j]) {
```

```
                            visited[j] = 1;
```

```
                            localQ.push_back(j);
```

```
                        }
```

```
    }  
    }  
    }  
}
```

```
#pragma omp critical  
for (int x : localQ)  
    q[rear++] = x;  
}
```

```
    front = oldRear;  
}  
}
```

```
int main() {
```

```
    int verts[5], start;  
    double seq[5], par[5], speedup[5], efficiency[5];
```

```
    int threads = omp_get_max_threads();
```

```
    cout << "Threads available: " << threads << "\n";
```

```
    for (int i = 0; i < 5; i++) {
```

```
        cout << "\nEnter vertices for graph "  
            << i + 1 << " (1-10000): ";
```

```
        cin >> verts[i];
```

```
        if (verts[i] < 1 || verts[i] > 10000) {
```

```

        cout << "Invalid input\n";
        return 1;
    }

    int max_possible = verts[i] * (verts[i] - 1) / 2;
    int edges = min(max_possible, verts[i] * 5);

    cout << "Generating " << edges << " edges\n";

    Graph g(verts[i]);

    srand(42 + i);
    g.generateEdges(edges);

    cout << "Enter starting vertex (0-"
        << verts[i] - 1 << "): ";

    cin >> start;

    if (start < 0 || start >= verts[i]) {
        cout << "Invalid vertex\n";
        return 1;
    }

    double t1 = omp_get_wtime();
    sequentialBFS(g, start);
    seq[i] = omp_get_wtime() - t1;

    double t2 = omp_get_wtime();
    parallelBFS(g, start);
    par[i] = omp_get_wtime() - t2;

```

```

    speedup[i] = seq[i] / par[i];
    efficiency[i] = speedup[i] / threads;
}

cout << "\n-----\n";
cout << "Graph\tVertices\tSeq Time\tPar Time\tSpeedup\tEfficiency\n";
cout << "-----\n";

for (int i = 0; i < 5; i++)
    cout << i + 1 << "\t"
        << verts[i] << "\t\t"
        << seq[i] << "\t"
        << par[i] << "\t"
        << speedup[i] << "\t"
        << efficiency[i] << "\n";

cout << "-----\n";

ofstream csv("bfs_results.csv");

csv << "graph,vertices,seq_time,par_time,speedup,efficiency\n";

for (int i = 0; i < 5; i++)
    csv << i + 1 << ", "
        << verts[i] << ", "
        << seq[i] << ", "
        << par[i] << ", "
        << speedup[i] << ", "
        << efficiency[i] << "\n";

```

```
csv.close();
```

```
cout << "\nResults saved to bfs_results.csv\n";
```

```
return 0;
```

```
}graph,vertices,seq_time,par_time,speedup,efficiency
```

```
1,3000,0.130681,0.202571,0.645112,0.161278
```

```
2,5000,0.357963,0.339465,1.05449,0.263623
```

```
3,7000,0.553271,0.646532,0.855751,0.213938
```

```
4,9000,0.563897,0.903934,0.623826,0.155956
```

```
5,10000,0.783011,1.1424,0.685409,0.171352
```

```
import csv
```

```
import matplotlib.pyplot as plt
```

```
vertices = []
```

```
seq_times = []
```

```
par_times = []
```

```
speedups = []
```

```
with open("bfs_results.csv") as f:
```

```
    reader = csv.DictReader(f)
```

```
    for row in reader:
```

```
        vertices.append(int(float(row["vertices"])))
```

```
        seq_times.append(float(row["seq_time"]))
```

```
        par_times.append(float(row["par_time"]))
```

```
        speedups.append(float(row["speedup"]))
```

```
labels = [str(v) for v in vertices]
```

```
x = range(len(labels))
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle("BFS: Sequential vs Parallel Performance", fontsize=14)

# Time comparison
ax1.plot(labels, seq_times, marker='o', label='Sequential', color='blue')
ax1.plot(labels, par_times, marker='s', label='Parallel', color='orange')
ax1.set_xlabel("Number of Vertices")
ax1.set_ylabel("Time (seconds)")
ax1.set_title("Execution Time")
ax1.legend()
ax1.grid(True)

# Speedup
ax2.plot(labels, speedups, marker='^', color='green', label='Speedup')
ax2.axhline(y=1.0, color='red', linestyle='--', label='Speedup = 1 (baseline)')
ax2.set_xlabel("Number of Vertices")
ax2.set_ylabel("Speedup (Seq Time / Par Time)")
ax2.set_title("Speedup")
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.savefig("bfs_speedup.png", dpi=150)
plt.show()
print("Graph saved as bfs_speedup.png")
```


BFS: Sequential vs Parallel Performance

